

7. Selection Sort on Random, Reverse and Sorted Arrays

Aim

To implement **Selection Sort** on random, reverse, and already sorted arrays and analyze its **time complexity**.

Algorithm

1. Start.
2. Take an array.
3. Find the smallest element in the unsorted part.
4. Swap it with the first element of the unsorted part.
5. Repeat until the complete array is sorted.
6. Perform the same operation for random, reverse, and sorted arrays.
7. Compare the results.
8. Stop.

Source Code

```
def selection_sort(arr):

    n = len(arr)

    for i in range(n):

        min_index = i

        for j in range(i + 1, n):

            if arr[j] < arr[min_index]:

                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

# Random Array

random_arr = [64, 25, 12, 22, 11]
```

```

# Reverse Array

reverse_arr = [50, 40, 30, 20, 10]

# Sorted Array

sorted_arr = [10, 20, 30, 40, 50]

print("Random Array:", selection_sort(random_arr))

print("Reverse Array:", selection_sort(reverse_arr))

print("Sorted Array:", selection_sort(sorted_arr))

```

Output

The screenshot shows a code editor with a file named 'main.py'. The code defines three arrays: 'random_arr' [64, 25, 12, 22, 11], 'reverse_arr' [50, 40, 30, 20, 10], and 'sorted_arr' [10, 20, 30, 40, 50]. It then prints the result of 'selection_sort' applied to each array. The output panel on the right shows the results: 'Random Array: [11, 12, 22, 25, 64]', 'Reverse Array: [10, 20, 30, 40, 50]', and 'Sorted Array: [10, 20, 30, 40, 50]'.

```

main.py Visualize Run ▶ Output
12
13     return arr
14
15
16 # Random Array
17 random_arr = [64, 25, 12, 22, 11]
18
19 # Reverse Array
20 reverse_arr = [50, 40, 30, 20, 10]
21
22 # Sorted Array
23 sorted_arr = [10, 20, 30, 40, 50]
24
25 print("Random Array:", selection_sort(random_arr))
26 print("Reverse Array:", selection_sort(reverse_arr))
27 print("Sorted Array:", selection_sort(sorted_arr))

```

Random Array: [11, 12, 22, 25, 64]
Reverse Array: [10, 20, 30, 40, 50]
Sorted Array: [10, 20, 30, 40, 50]

Complexity Analysis

Array Type	Time Complexity
Random Array	$O(n^2)$
Reverse Array	$O(n^2)$
Sorted Array	$O(n^2)$

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

Selection Sort always performs almost the same number of comparisons, even when the array is already sorted.

Result

Thus, **Selection Sort** was successfully implemented for random, reverse, and sorted arrays. The time complexity for all three cases is **$O(n^2)$** and the space complexity is **$O(1)$** .